Prova-T (peso 2,5): Softwares de Sistemas, resolvida e comentada

Arquivo: `SB_Prova_Sistemas-EU.pdf` . CIC0104 Software Básico, semestre 21/2, UnB/CIC, Turma A.
Estrutura: Parte 1 com 22 itens V/F/NS (7,0 pontos, 0,32 cada) e Parte 2 com 4 questões discursivas (1,0 + 1,0 + 1,0 + 2,0).

Regras da prova (importam para a estratégia):

- Cada item objetivo é marcado com **F**, **V** ou **NS** (não sei).
- **Duas respostas erradas anulam uma certa.**
- **Resposta em branco conta como errada.** NS presumivelmente vale zero e não desconta.
- **Todo item marcado F precisa de justificativa.**

Sobre as respostas do estudante: a versão "EU" tem uma camada visível com as respostas do próprio estudante: a coluna V/F e a lista de justificativas em fonte Arial. Por baixo há um texto oculto, herdado do PDF original de outro aluno (`SB_Prova_Sistemas.pdf`). Considerei **somente o que aparece visualmente** na versão EU.

Assuntos cobertos:

1. Arquitetura de von Neumann: componentes, ciclo de busca e execução, gargalo, localidade de referência.
2. Código absoluto x relocação estática.
3. Carregadores: absoluto e de relocação.
4. Montador: entrada/saída, estrutura lógica, passos, síntese/otimização.
5. Ligador: etapas, saída, tipos (Linkage Editor, Linking Loader/Binder, ligação dinâmica/DLL).
6. Sistemas operacionais: tipos (batch, time sharing etc.), escalonamento, deadlock, starvation, região crítica.
7. Gerência de memória real e virtual: partições, swapping, paginação, segmentação, paginação-segmentação, page fault.

Referências aos slides. O número do slide é o que aparece impresso no canto do slide.

- `7-teoria-1 de 2.pdf` : o slide N fica na página N do PDF.
- `7-teoria-2 de 2.pdf` : os slides vão de 34 a 50 (slide N na página N−32). As páginas 19 a 27 tratam de DLL e não têm número.
- `8-so-red-1 de 2.pdf` : slide N na página N.

- `8-so-red-2 de 2.pdf` : slides de 15 a 29 (slide N na página N−13).
- `9-boot-so-red.pdf` : slide N na página N.

Parte 1: questões objetivas

Questão 1

Enunciado:

> A arquitetura de von Neumann é caracterizada por CPU, memória e periféricos, sendo a CPU composta por unidade lógica e aritmética, unidade de controle, registradores e contador de programas – PC. Na memória RAM são armazenados dados e programas.

Resposta do estudante: F. Justificativa: "A arquitetura de von Neumann é caracterizada por CPU, memória, periféricos e barramentos. A memória armazena dados e programas." **CERTA.**

Resposta correta: F (provável).

Comentário: o slide 5 de `7-teoria-1` caracteriza a arquitetura por **quatro** elementos:

- **CPU:** ULA, UC, registradores e PC.
- **Memória:** dados e programas.
- **Periféricos de E/S:** podem ser mapeados na memória.
- **Barramentos:** dados, endereço e controle.

O enunciado omite os **barramentos**. Ele também fala em "memória RAM", mas o slide diz apenas "memória": o programa também pode estar em ROM, como o código absoluto do boot. A justificativa do estudante aponta exatamente o elemento que falta.

Pegadinha: o professor costuma copiar a lista do slide e **tirar um item**. Decore as listas completas: 4 componentes da arquitetura, 4 componentes da CPU, 3 tipos de barramento.

Incerteza: a falsidade aqui vem de uma **omissão**. Pelo padrão do professor (e pelo que a turma marcou), F é a resposta esperada. Ainda assim, isso depende da convenção dele de considerar "incompleto = falso".

Questão 2

Enunciado:

> A execução é rápida na arquitetura de von Neumann porque dados e instruções são armazenados em blocos de memória adjacentes devido ao fenômeno de localidade de referência.

Resposta do estudante: V. **CERTA.**

Resposta correta: V.

Comentário: o enunciado é praticamente uma transcrição do slide 7 de `7-teoria-1` : "Execução rápida porque dados e instruções são armazenados em blocos de memória adjacentes → Localidade de referência".

Como os dados ficam contíguos, os acessos tendem a se concentrar em regiões próximas. É isso que cache, buffers e paginação exploram.

Dica: uma leitura muito literal poderia ver uma "inversão de causa", já que a localidade é uma propriedade do acesso e não a causa do armazenamento adjacente. Mesmo assim, o texto é o do slide, e o professor considera V.

Questão 3

Enunciado:

> A função básica do ligador é ligar os módulos e resolver as referências externas, seguindo as seguintes etapas: ler e relocar de forma estática o módulo objeto P; atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos; concatenar P aos módulos anteriores; e resolver as referências externas em P.

Resposta do estudante: V. **CERTA.**

Resposta correta: V.

Comentário: as etapas e a função básica estão nos slides de `7-teoria-2` :

- **Etapas (slide 40):** (1) ler e relocar o módulo objeto P; (2) atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos; (3) concatenar P aos módulos anteriores; (4) resolver as referências externas em P.

- **Função básica (slide 34):** ligação de módulos e resolução de referências externas.
- **"De forma estática" está correto:** o ligador faz **relocação estática**, somando a constante de relocação k ao endereço relativo (slides 38, 39 e 43). Para o módulo P2, k = size(P1); para P3, k = size(P1, P2).

Dica: veja também o ligador de dois passos (slide 42). No passo 1 ele reloca e monta a tabela global de símbolos. No passo 2 ele resolve as referências externas.

Questão 4

Enunciado:

> A redução do gargalo de von Neumann entre CPU e periféricos de I/O pode ser obtida com o uso de memória **cache** e processadores **pipelining** com múltiplos estágios.

Resposta do estudante: F. Justificativa: "A solução apresentada é para a redução do gargalo de von Neumann entre CPU e memória." **CERTA.**

Resposta correta: F.

Comentário: o slide 8 de `7-teoria-1` separa dois gargalos:

Gargalo	Técnicas de redução
CPU x memória	memória cache; processadores com múltiplos estágios (pipelining)
Memória x periféricos de E/S	blocagem e bufferização; DMA (acesso direto à memória); canais de controle de dispositivos de E/S

Cache e pipelining atacam o gargalo CPU x memória, não o de E/S.

Pegadinha: a tabela acima é cobrada em quase toda prova. A troca entre as duas colunas é a pegadinha clássica.

Questão 5

Enunciado:

> Ao se ligar um computador, pressionando o botão de **power on**, um carregador absoluto armazenado em memória volátil é ativado para iniciar o processo de carga do sistema operacional.

Resposta do estudante: F. Justificativa: "O carregador absoluto é armazenado em memória não volátil." **CERTA.**

Resposta correta: F.

Comentário: o slide 15 de `7-teoria-1` define o carregador absoluto assim:

- Inicia a carga do S.O.
- É código absoluto.
- Fica **armazenado em memória não volátil, em posição fixa e conhecida** (ROM, flash).
- É ativado pela "ação do operador ao apertar o botão".
- Em mainframe recebe o nome de IPL.

Se a memória fosse volátil, o código se perderia ao desligar a máquina. Veja também o slide 6 de `9-boot-so-red` : a CPU executa a ROM BIOS, e no endereço FFFF:0000 há um `jmp` .

Pegadinha: a troca de "não volátil" por "volátil" é a única alteração em relação ao slide.

Questão 6

Enunciado:

> As otimizações algorítmicas de código que podem ser realizadas na síntese do código são de tempo de execução ou de alocação de memória

Resposta do estudante: marcou **"F/NS"**. Justificativa: "A otimização algorítmica do código pode ser realizada pelo montador." **PARCIALMENTE ERRADA.**

- **A marcação é inválida.** As instruções exigem F, V ou NS. "F/NS" tende a ser tratado como resposta errada ou anulada.
- **A justificativa está errada.** Ela não aponta o erro do enunciado e ainda afirma o contrário do slide: a síntese do montador **não** faz otimização algorítmica.

Resposta correta: F (provável).

Comentário: o slide 31 de `7-teoria-1` diz que a síntese "otimiza código (opcional). Otimiza o código intermediário gerado pela análise semântica. **A otimização é do código e não das funções**

implementadas ao nível do código do programador".

Otimização **algorítmica** é justamente mexer no algoritmo, ou seja, nas funções que o programador implementou. A síntese só faz otimizações do código intermediário. Ela não reescreve o algoritmo.

Justificativa modelo: "Na síntese, a otimização é do código intermediário, e não algorítmica (das funções implementadas pelo programador)."

Incerteza: alguém poderia achar que o professor só quis classificar as otimizações (tempo x espaço) e que o item seria V. O colega da versão original e o aluno de `ProvaSB_teoria.pdf` marcaram V. Ainda assim, a nota do slide 31 parece ter sido escrita exatamente para essa pegadinha. Fico com **F**, mas com dúvida moderada.

Questão 7

Enunciado:

> Carregador de relocação estática lê o tamanho do programa a carregar; solicita ao sistema operacional (S.O.) alocar um bloco contíguo de memória com tamanho suficiente para conter o programa; carrega o programa a partir do endereço dado pelo S.O., reloca os endereços relocáveis; e inicializa o PC com o **entrypoint** do programa.

Resposta do estudante: V. **CERTA.**

Resposta correta: V.

Comentário: o slide 49 de `7-teoria-2` descreve o carregador de relocação em ambiente monousuário. Ele carrega na RAM um executável que está em disco e:

1. lê o tamanho do programa e o entrypoint;
2. solicita ao gerente de memória um bloco contíguo;
3. carrega o programa a partir do endereço dado pelo S.O., relocando os endereços relocáveis;
4. transfere o controle a partir do entrypoint.

"Inicializa o PC com o entrypoint" é o mesmo que transferir o controle.

Dica: o slide pergunta "qual o problema com esse esquema?". A resposta é que exige um **bloco contíguo**, o que gera fragmentação. Além disso, depois de carregado, o programa não pode mais ser movido, porque a relocação é estática.

Questão 8

Enunciado:

> Código absoluto foi utilizado no início da computação com sistemas monousuário e não é mais utilizado devido a que os sistemas atuais são multiprogramados ou multiprocessados.

Resposta do estudante: F. Justificativa: "Dependendo do sistema, o código absoluto ainda pode ser utilizado atualmente." **CERTA.**

Resposta correta: F.

Comentário: o slide 13 de `7-teoria-1` diz que o código absoluto (posições e endereços fixos e reais) ainda é usado "**em memória ROM em procedimentos de boot**", como no endereço FFFF:0000 da ROM BIOS. O carregador absoluto, que inicia a carga do S.O., é código absoluto (slide 15).

A justificativa do estudante está certa, mas é genérica. Uma justificativa mais forte citaria o **boot/ROM BIOS e o carregador absoluto**.

Questão 9

Enunciado:

> Do ponto de vista lógico, em geral um montador é estruturado em um pré-processamento (retira comentários, expande macro e faz a montagem condicional), passo 1 (análises léxica, sintática e semântica) e passo 2 (otimização e geração de código absoluto).

Resposta do estudante: F. Justificativa: "O montador gera um código objeto e não um código absoluto." **CERTA.**

Resposta correta: F.

Comentário:

- **Saída do montador:** pelos slides 17, 18 e 31 de `7-teoria-1` , o montador gera **código objeto relocável**, não código absoluto.
- **Pré-passo:** a descrição bate com o slide 26 (montagem condicional, macros, código limpo sem comentários).

- **Passos:** a divisão em passo 1 (análise) e passo final (síntese) está de acordo com os slides 18 e 25. O erro está só em "código absoluto".

Pegadinha: "código absoluto" no lugar de "código objeto relocável" aparece em várias questões, inclusive na versão de `ProvaSB_teoria.pdf`. O montador **sempre** gera código objeto relocável, acompanhado de uma tabela de endereços relocáveis e de uma lista de referências externas.

Questão 10

Enunciado:

> Em um S.O. do tipo **batch** a melhor gerência de memória real é paginação-segmentação.

Resposta do estudante: F. Justificativa: "Paginação-segmentação é uma abordagem de gerência de memória virtual." **CERTA** (com a ressalva abaixo).

Resposta correta: F (provável).

Comentário:

- **Onde o slide coloca paginação-segmentação:** no slide 23 de `8-so-red-2`, ela aparece sob o título "**Gerência de Memória Virtual**: Segmentação-paginação". É o esquema mais sofisticado, com dois níveis de tabela (segmentos e páginas), e não uma gerência de memória **real** característica de batch.
- **O que combina com batch:** batch é processamento em lote, com jobs longos e **CPU-intensivos** sem interação. O casamento típico é com **partições**: multiprogramação com alocação contígua (slides 15 e 17).
- **O que combina com time sharing:** é com time sharing que se casa o swapping, porque troca processos por time slice (slide 18).

Incerteza: os slides não trazem uma tabela explícita do tipo "tipo de S.O. → melhor gerência". A resposta vem da associação entre os slides 15, 17, 18 e 23 e do raciocínio sobre a carga de trabalho. Além disso, o slide 15 lista paginação-segmentação na tabela "Abordagens de Gerência de Memória" (multiprogramação, não contígua), o que deixa a fronteira real/virtual um pouco confusa. Mesmo assim, F é a resposta mais defensável.

Questão 11

Enunciado:

> Em um S.O. do tipo time **sharing** como as transações são CPU intensiva, a melhor gerência de memória real é de partições.

Resposta do estudante: F. Justificativa: "Sistema operacional do tipo batch." **CERTA** (a justificativa está correta, mas curta demais).

Resposta correta: F.

Comentário:

- **Natureza das transações:** em time sharing elas são **interativas**, curtas e voltadas a E/S (usuário no terminal), e não CPU-intensivas. CPU-intensivo é o perfil do **batch**, e é isso que o estudante quis dizer.
- **Gerência adequada:** a que combina com time sharing é o **swapping**, que "troca processos via interrupção (time slice)" com escalonamento por prioridade (slide 18 de `8-so-red-2`).
- **Por que não partições:** partições combinam com batch (slide 17).

Justificativa modelo: "Em time sharing as transações são interativas (E/S intensivas), não CPU intensivas; isso caracteriza o batch. A gerência típica de time sharing é o swapping (time slice)."

Dica: a combinação **time sharing + time slice + swapping** e a combinação **batch + CPU-intensivo + partições** são o núcleo da discursiva 4.

Questão 12

Enunciado:

> Gerência de memória baseada em páginas minimiza a fragmentação interna, favorece a proteção de código e a fragmentação externa.

Resposta do estudante: F. Justificativa: "O modo de gerência descrito é o baseado em segmentação." **CERTA.**

Resposta correta: F.

Comentário:

	Paginação	Segmentação
Tamanho do bloco	fixo (alguns KB)	variável (ligado à lógica do programa)
Fragmentação interna	**existe** (última página parcialmente usada)	praticamente não existe
Fragmentação externa	**não existe** (blocos fixos)	**existe** (blocos variáveis)
Proteção de código	facilita (slide 20)	"facilita em muito" (slide 22)
Cálculo do endereço real	**concatenação** página real ‖ deslocamento	**soma** (não pode concatenar), mais caro

O enunciado descreve a segmentação, não a paginação, e a justificativa do estudante acerta isso. Na versão de `ProvaSB_teoria.pdf`, o mesmo item fala em "segmentos". Fontes: slides 13, 20, 21 e 22 de `8-so-red-1` e `8-so-red-2`.

Pegadinha: o professor troca "páginas" por "segmentos" entre as versões da prova. Associe **tamanho fixo → fragmentação interna** e **tamanho variável → fragmentação externa**.

Questão 13

Enunciado:

> Ligadores são dos tipos: **Linkage Editor** (gera um arquivo binário que pode ser reusado) ou **Linking Loader**/**Binder** (carrega o código gerado na memória sem gerar o arquivo que possa ser reusado), ou **carregador de função** de biblioteca de ligação dinâmica (tipo DLL).

Resposta do estudante: V. **CERTA.**

Resposta correta: V (provável).

Comentário: a classificação vem de `7-teoria-2`:

- **Linkage Editor (slide 48):** faz a ligação antes da carga e grava o programa em arquivo (o "módulo de carga"), que pode ser reexecutado. Na carga, só é preciso relocar.
- **Linking Loader ou Binder (slide 48):** depois da ligação, carrega o programa na memória. "Não é gerado um arquivo executável."

- **Dynamic linking (slide 50):** a ligação ocorre em tempo de execução. A rotina é carregada e ligada quando é chamada pela primeira vez. O exemplo do slide é a DLL.

Incerteza: o termo "carregador de função" não aparece literalmente nos slides. Os slides de DLL atribuem a carga ao carregador/ligador em tempo de execução, então o item parece V.

Questão 14

Enunciado:

> Na arquitetura de von Neumann o processamento é sequencial e segue o ciclo: busca – decodificação – busca de operandos – execução. O carregador carrega o programa na memória, atribui a PC o valor do **entry point** (endereço de uma instrução) e inicia o ciclo.

Resposta do estudante: F. Justificativa: "A cada ciclo de busca, PC é incrementado."
PROVAVELMENTE ERRADA.

Resposta correta: V (provável, mas é o item mais incerto da prova).

Comentário: o slide 7 de `7-teoria-1` diz:

- "Processamento é seqüencial";
- "Ciclo busca – decodificação – busca de operandos – execução (fetch – execute cycle) e PC";
- "PC ← Entry point";
- "PC é incrementado a cada ciclo de busca".

Tudo o que o enunciado **afirma** está no slide, sem nenhuma alteração. A justificativa do estudante não aponta um erro: ela só acrescenta um fato (o incremento do PC) que o enunciado não nega. Omitir um detalhe que **não faz parte da afirmação** não torna o item falso.

Aqui a situação é diferente da Q1. Lá, a frase "é caracterizada por" pretendia dar a lista completa e deixou um componente de fora. Aqui, a frase não diz que o PC "não é incrementado".

Incerteza: depende da convenção do professor sobre omissões. Se ele considera falso todo enunciado incompleto, a resposta seria F. Contudo, o texto bate com o slide, e na outra versão (`ProvaSB_teoria.pdf` , questão 2) o enunciado equivalente tem a mesma omissão e aparece como V. Por isso considero **V** mais provável.

Dica de prova: se um item reproduz o slide sem nenhuma troca de palavra, ele quase sempre é V. Para justificar F, aponte uma afirmação **errada**, não uma informação que falta.

Questão 15

Enunciado:

> O advento do código relocável permitiu o uso de bibliotecas de código objeto como é o caso do **bytecode** Java que roda em qualquer computador com uma JVM Java instalada.

Resposta do estudante: NS. Não perdeu ponto, mas também não ganhou.

Resposta correta: F (provável).

Comentário:

- **A primeira parte é verdadeira.** O slide 14 de `7-teoria-1` diz que a relocação estática "VIABILIZA A CONSTRUÇÃO DE BIBLIOTECAS DE CÓDIGO OBJETO". O slide 23 repete: "É a existência da relocação que possibilita o uso de bibliotecas de código objeto".
- **O exemplo está errado.** O bytecode Java **não** é código objeto relocável (código de máquina com tabela de endereços relocáveis). É um código intermediário para uma **máquina virtual**. Ele "roda em qualquer computador" por causa da **JVM**, que interpreta ou compila sob demanda, e não por causa da relocação estática.

Justificativa modelo: "O bytecode Java não é código objeto relocável; sua portabilidade se deve à máquina virtual (JVM), não à relocação."

Incerteza: o colega de `ProvaSB_teoria.pdf` marcou V, aceitando o exemplo. O erro de conceito no exemplo, porém, é claro. Fico com F, com incerteza moderada.

Pegadinha: frase verdadeira + exemplo errado = falso. Leia até o fim.

Questão 16

Enunciado:

> Para maximizar o **throughput** e evitar **deadlock** um processo só pode entrar na fila de prontos se já estiver com todos os recursos que necessitar.

Resposta do estudante: F. Justificativa: "A solução descrita não maximiza o throughput." **CERTA.**

Resposta correta: F.

Comentário:

- **Evita deadlock?** Sim. Exigir todos os recursos antes de executar quebra a condição de "posse e espera" (hold and wait) e é uma técnica clássica de prevenção.
- **Maximiza o throughput?** Não, o efeito é o oposto. Os recursos ficam presos por processos que ainda não precisam deles, a utilização cai, os processos esperam mais e pode haver até **starvation** de quem precisa de muitos recursos.
- **Cobertura nos slides:** é fraca. O slide 10 de `8-so-red-1` (gerenciamento de alocação de recursos, região crítica, "mecanismos podem ser implementados com o objetivo de evitar situações de deadlock") só trata do tema por cima. As condições de deadlock são conteúdo geral de S.O.

Pegadinha: o item junta dois objetivos ("maximizar throughput **e** evitar deadlock"). Basta um ser falso para o item ser F.

Questão 17

Enunciado:

> Região crítica é o trecho do código que objetiva garantir a exclusão mútua de processos.

Resposta do estudante: V. **CERTA** (provável).

Resposta correta: V (provável).

Comentário:

- **O que é região crítica:** é o trecho de código em que o processo acessa um recurso compartilhado. Esse trecho é delimitado e controlado para que apenas um processo o execute por vez (exclusão mútua).
- **O que diz o slide:** o slide 10 de `8-so-red-1` associa a região crítica ao controle de alocação de recursos pelo S.O. Se o recurso está alocado, o processo é bloqueado. Se está livre, é alocado e o controle volta ao processo.
- **Origem histórica do termo:** "região crítica" (Brinch Hansen) é também uma construção de linguagem criada justamente para garantir exclusão mútua.

Incerteza: dá para argumentar que a região crítica é o trecho que **exige** exclusão mútua, e não o que a **garante**. Quem garante seriam semáforos, monitores etc. O colega da versão original marcou F

com esse argumento. O enunciado, porém, parece ter sido escrito como definição V, e o aluno de `ProvaSB_teoria.pdf` também marcou V.

Questão 18

Enunciado:

> Se um processo de posse da CPU solicitar acesso a uma página não disponível na memória será criado um programa de canal para carregar a página em uma **page frame** e o processo ficará bloqueado.

Resposta do estudante: V. **CERTA.**

Resposta correta: V.

Comentário: é um **page fault** (falta de página). A página precisa ser trazida do disco, e isso é uma operação de E/S. Pelo slide 9 de `7-teoria-1`, uma operação de E/S com interrupção segue estes passos:

1. Gera uma chamada ao Supervisor.
2. O Supervisor **cria um programa para o canal de E/S** e as controladoras.
3. O processo é **bloqueado** em uma fila específica.
4. Ao terminar, o canal gera uma interrupção.
5. O controlador de interrupções desbloqueia o processo, que vai para a fila de prontos.

O slide 11 de `8-so-red-1` confirma que as operações de um canal são definidas em um programa para o canal. Pelo slide 20 de `8-so-red-2`, se não houver frame livre, um algoritmo como o LRU escolhe a página a remover.

Dica: na versão de `ProvaSB_teoria.pdf` há uma pergunta extra: qual mecanismo tira o processo da fila de bloqueados e o põe na de prontos? O slide 9 diz que é o **controlador de interrupções**, acionado pela interrupção do canal. (O outro aluno respondeu "escalonador".)

Questão 19

Enunciado:

> Sistemas de escalonamento procuram maximizar o **throughput** e minimizar o **response time** e o **turnaround time.** Por isso um bom escalonador deve ser não preemptivo.

Resposta do estudante: F. Justificativa: "Um bom escalonador pode ser preemptivo ou não preemptivo." **CERTA** (a justificativa é aceitável, mas fraca).

Resposta correta: F.

Comentário:

- **A primeira frase está no slide.** O slide 9 de `8-so-red-1` diz: "Sistemas de escalonamento procuram maximizar o throughput e minimizar o turnaround time e o response time".
- **O erro é a conclusão.** Para minimizar o **response time** é preciso **preempção** (time slice). Um escalonador não preemptivo deixa um processo longo monopolizar a CPU, e o tempo de resposta dos interativos explode.
- **Sobre a justificativa do estudante:** ela é literalmente o slide ("Escalonamento pode ser preemptivo ou não preemptivo") e derruba o "deve ser". Seria mais forte explicar **por que** a conclusão é falsa.

Justificativa modelo: "Minimizar response time exige preempção (time slice); não preemptivo deixa processos longos monopolizarem a CPU."

Questão 20

Enunciado:

> Um escalonamento de processos adequado para um S.O. multiprocessado é o **round-robin** pois permite evitar **deadlock.**

Resposta do estudante: F. Justificativa: "Round-robin evita starvation." **CERTA.**

Resposta correta: F.

Comentário:

- **O que o round-robin resolve:** todo processo da fila de prontos recebe uma fatia de tempo (time slice) em rodízio, então nenhum fica sem CPU indefinidamente. Ou seja, ele evita **starvation**.
- **O que ele não resolve: deadlock** é espera circular por **recursos**, e o escalonamento da CPU não resolve isso.
- **Cobertura nos slides:** o round-robin aparece no slide 9 de `8-so-red-1` .

Pegadinha: na versão de `ProvaSB_teoria.pdf`, o item diz "pois permite evitar **starvation**", e nesse caso seria V. Atenção à troca **deadlock ↔ starvation**.

Questão 21

Enunciado:

> Um ligador, a partir de arquivos de módulos objetos relocáveis, resolve as referências externas e gera um arquivo contendo os códigos objetos ligados, e com a indicação dos endereços relocáveis. Esse arquivo, união dos códigos objetos, não é executável.

Resposta do estudante: F. Justificativa: "O arquivo gerado pelo ligador é executável, alguns ligadores apenas não geram esse arquivo." **CERTA.**

Resposta correta: F.

Comentário: o slide 34 de `7-teoria-2` define a saída do ligador como "**arquivo binário com código de máquina relocável executável**" mais os endereços relocáveis. A ressalva do estudante também está correta: o Linking Loader/Binder não gera arquivo, apenas carrega o programa na memória (slide 48).

Pegadinha: a versão de `ProvaSB_teoria.pdf` termina com "é executável" e seria V. O professor alterna o final entre as versões.

Questão 22

Enunciado:

> Um montador é um programa que converte um programa em código Assembly em um código objeto relocável (código de máquina), se o programa Assembly estiver correto, e cria um arquivo binário com código de máquina relocável, contendo uma tabela (ou lista) de endereços relocáveis e uma lista de símbolos não resolvidos pelo montador.

Resposta do estudante: V. **CERTA.**

Resposta correta: V.

Comentário: o item é a transcrição do slide 17 de `7-teoria-1`. O montador converte Assembly em código objeto relocável, se o programa estiver correto. A saída tem três partes:

- arquivo binário com código de máquina relocável;
- tabela (ou lista) de endereços relocáveis;
- lista de referências externas (símbolos não resolvidos).

A nota do slide acrescenta: "Se houver erro de sintaxe, o montador não gera o código".

Resumo da Parte 1

#	Estudante	Gabarito	Situação
1	F	F (provável)	certa
2	V	V	certa
3	V	V	certa
4	F	F	certa
5	F	F	certa
6	F/NS	F (provável)	marcação inválida, justificativa errada
7	V	V	certa
8	F	F	certa
9	F	F	certa
10	F	F (provável)	certa
11	F	F	certa
12	F	F	certa
13	V	V (provável)	certa
14	F	V (provável)	**errada**
15	NS	F (provável)	não sabia
16	F	F	certa

#	Estudante	Gabarito	Situação
17	V	V (provável)	certa
18	V	V	certa
19	F	F	certa
20	F	F	certa
21	F	F	certa
22	V	V	certa

Estimativa da nota na Parte 1. Cada item vale 0,32, e dois erros anulam um acerto.

- **Se a Q6 for aceita como F:** 20 acertos, 1 erro e 1 NS, ou seja, (20 − 0,5) × 0,32 ≈ **6,24**.
- **Se a Q6 for considerada errada:** 19 acertos e 2 erros, ou seja, (19 − 1) × 0,32 ≈ **5,76**.

Parte 2: questões discursivas

Discursiva 1 (1,0)

Enunciado:

> Caracterize o que é (0,5) e o que causa localidade de referência (0,5).

Resposta do estudante:

> "Localidade de referência é a tendência do processador acessar várias vezes as mesmas áreas de memória em um curto período de tempo. Pode ser causada pelo acesso ao mesmo dado várias vezes, ou pelo acesso à vários dados em endereços muito próximos."

Avaliação: PARCIALMENTE CERTA (estimativa de 0,5 a 0,75).

- **"O que é":** está bom, com a ideia de concentração dos acessos em poucas regiões num intervalo curto.
- **"O que causa":** está fraco. O estudante só repete os **dois tipos** de localidade (temporal e espacial), sem dizer **o que no programa ou na arquitetura** os provoca.

Resposta modelo:

O que é (0,5). Localidade de referência é a tendência de um programa, durante um intervalo de tempo, concentrar seus acessos à memória em um conjunto pequeno de endereços. Ela tem duas formas:

- **Localidade temporal:** um endereço acessado agora tende a ser acessado de novo em breve.
- **Localidade espacial:** depois de acessar um endereço, o programa tende a acessar endereços vizinhos.

Na arquitetura de von Neumann, é isso que torna a execução rápida. Dados e instruções ficam em blocos adjacentes de memória (slide 7 de `7-teoria-1`), e mecanismos como **cache**, **buffers** e **paginação com LRU** exploram essa propriedade para reduzir o gargalo de von Neumann.

O que causa (0,5):

- **Execução sequencial das instruções.** O PC é incrementado a cada ciclo de busca, então as instruções seguintes estão nos endereços seguintes (espacial). A exceção são os desvios.
- **Laços (loops).** As mesmas instruções e as mesmas variáveis (contadores, acumuladores) são acessadas repetidas vezes (temporal).
- **Estruturas de dados contíguas.** Vetores, matrizes e registros são percorridos em endereços consecutivos (espacial).
- **Pilha e variáveis locais.** Chamadas de sub-rotina usam sempre a região do topo da pilha (temporal e espacial).
- **Organização do código pelo montador e pelo ligador.** Código e dados de um módulo ficam em blocos contíguos, e os módulos são concatenados.

Dica: sempre cite os dois tipos (temporal e espacial) e dê causas concretas (laços, vetores, execução sequencial).

Discursiva 2 (1,0)

Enunciado:

> Quais os tipos de carregadores existentes? (0,5) Cite o instante em que os endereços são relocados (0,25) e o que é carregado em cada um dos tipos de carregador que citar (0,25).

Resposta do estudante:

> "Carregador Absoluto: Inicia a carga do sistema operacional.
> Carregador de Relocação: Carrega na memória RAM o programa executável contido em disco."

Avaliação: PARCIALMENTE CERTA (estimativa de cerca de 0,75).

- **Os dois tipos estão certos (0,5)**, assim como o que cada um carrega (0,25).
- **Faltou o instante da relocação (0,25).**

Resposta modelo:

Carregador	Instante da relocação	O que carrega	Detalhes (slides)
Carregador absoluto	**Não há relocação.** O código é absoluto, com endereços fixos e reais definidos antes (na tradução).	O **sistema operacional**, ou seja, inicia a carga do S.O. (bootstrap/IPL).	Fica em memória **não volátil** (ROM, flash), em posição fixa. É ativado pelo botão power on. Lê registros com endereço de carga, número de palavras e dados, e no fim faz `transferto` para o endereço de carga. (slides 15 e 16 de `7-teoria-1`)
Carregador de relocação (relocação estática)	**No momento da carga.** O S.O. informa o endereço inicial do bloco (α ou k), e o carregador soma essa constante a cada endereço relocável listado na tabela de endereços relocáveis.	Um **programa executável em disco**, levado para a RAM.	Lê o tamanho e o entrypoint, pede um bloco contíguo ao S.O., carrega e reloca, e transfere o controle para o entrypoint. (slide 49 de `7-teoria-2` ; slide 14 de `7-teoria-1` : endereço efetivo = offset + α)

Variações que podem render pontos extras:

- **Linking Loader / Binder (slide 48 de `7-teoria-2`):** faz ligação, relocação e carga no mesmo instante (tempo de carga), sem gerar arquivo executável.
- **Ligação/carga dinâmica, como a DLL (slides 50 a 53):** a rotina de biblioteca é carregada e relocada em **tempo de execução**, na primeira chamada ou na carga do programa. A relocação usa a tabela de fixup.
- **Relocação dinâmica com registrador de base (slide 19 de `8-so-red-2`):** a relocação acontece **durante a execução**, a cada acesso (ER = EL + RegBase, via DAT).

Dica: a pergunta tem três partes com pontos separados. Responda cada uma explicitamente, de preferência em tabela. O estudante perdeu a parte do **instante**.

Discursiva 3 (1,0)

Enunciado:

> Apresente um exemplo e explique como é realizado o mapeamento de um endereço lógico em um endereço real em um ambiente paginado.

Resposta do estudante:

> "Um endereço real é a soma do endereço lógico com um registrador base. Em um ambiente paginado, além dessa soma, é preciso acessar uma tabela de mapeamento de páginas, que contém uma entrada para cada página física do programa, para compor o endereço real final."

Avaliação: PARCIALMENTE CERTA / FRACA (estimativa de 0,25 a 0,5).

- **Não apresentou exemplo**, que é exigido pelo enunciado.
- **Confundiu dois esquemas.** "Endereço real = endereço lógico + RegBase" é a **relocação dinâmica por registrador de base** (slide 19), um esquema contíguo. Na paginação, o RegBase aponta para o **início da tabela de páginas** e é somado ao **número da página**, não ao endereço lógico inteiro.
- **Não explicou que o endereço lógico se divide em página e deslocamento.**
- **Não disse que o endereço real é formado por concatenação** do frame com o deslocamento.
- **Mérito:** ele mencionou a tabela de mapeamento de páginas com uma entrada para cada página física.

Resposta modelo:

Figura de referência. O slide 21 de `8-so-red-2` mostra o seguinte:

- O endereço lógico tem dois campos: o número da página virtual (rótulo "e−d", os bits altos) e o deslocamento **d**.
- O registrador **RegBase** é somado (+) ao número da página para localizar a entrada na **tabela de mapeamento de páginas**.
- Cada entrada tem os campos **r**, **s** e **p**. O campo **p** é a página real (frame). Pelo contexto do slide 20, r provavelmente é o bit de referência usado pelo LRU, e s um bit de status/presença.
- O endereço real é formado por **p** e **d** lado a lado, e é com ele que se acessa a memória.

Explicação:

1. A memória física é dividida em **page frames** de tamanho fixo, potência de 2 (alguns KB). O programa é dividido em páginas do mesmo tamanho (slide 20).
2. O endereço lógico gerado pela CPU é dividido em **número da página virtual** (bits mais altos) e **deslocamento d** (bits mais baixos). Se a página tem 2^n bytes, d tem n bits.
3. O **DAT** (Dynamic Address Translator, em hardware) soma o **RegBase** (início da tabela de páginas do processo) ao **número da página**. Isso localiza a entrada correspondente na **tabela de mapeamento de páginas**.
4. A entrada fornece o **número da página real (frame) p**. Se a página não estiver na memória, ocorre **page fault**: interrupção, programa de canal para trazer a página do disco e processo bloqueado (ver Q18).
5. O **endereço real é a concatenação p || d**, e não uma soma. Como os frames são alinhados em múltiplos do tamanho da página, basta justapor os bits. É mais barato que somar. Já na segmentação, os blocos têm tamanho variável, então é preciso **somar** (slide 22).

Exemplo: páginas de 4 KB = 2^{12} bytes, então d tem 12 bits (3 dígitos hexadecimais).

- Endereço lógico: **0x2ABC**. Página virtual = 0x2, deslocamento d = 0xABC.
- Tabela de páginas do processo, com início em RegBase:

Página virtual	Página real (frame)	Presente?
0	7	sim
1	3	sim
2	**5**	sim
3	–	não (page fault)

- Endereço da entrada consultada: RegBase + 2. A entrada informa o frame p = 5.
- Endereço real = p ‖ d = **0x5** ‖ **ABC** = **0x5ABC**.
- Se o endereço lógico fosse 0x3010, a página 3 não estaria presente e ocorreria **page fault**.

Dica: o slide 21 pergunta qual operador se aplica a p e d. A resposta é **concatenação**, porque os frames têm tamanho fixo e alinhado. Lembre de **diferenciar** registrador de base (slide 19, soma do endereço inteiro) de paginação (slide 21, RegBase + página na tabela, depois concatenação).

Discursiva 4 (2,0)

Enunciado:

> Coloque-se na posição de projetista de S.O. Indique qual é a aplicação característica (**batch**, **real time**, **time sharing**, rede ou distribuído), forma de carregamento de programas na memória (contíguos ou não), tipo (monoprogramado, multiprogramado ou multiprocessado), e estrutura (hierárquica, máquina virtual, multiprocessado, distribuído ou orientado a objetos). Apresente e justifique a **política de processos adequada** (0,5) indicando como ela evita **deadlock** (0,5) e **starvation** (0,5), e a **política de gerência de memória real**. (0,5) (2,0)

Resposta do estudante:

> "Time sharing, alocação de memória contígua, multiprogramado.
> Gerência de memória real: Swapping
> Política de processos: Time-slice
> No swapping é ideal ter alocação de memória contígua e um sistema multiprogramado, funciona a base de time-slice com troca de processos via interrupção. Com isso, a aplicação característica ideal é o time sharing.
> Para evitar deadlock, os processos devem liberar os recursos quando não for mais necessário.
> Para evitar starvation, o swapping escalona os processos por prioridade."

Avaliação: PARCIALMENTE CERTA (estimativa de 0,75 a 1,0 de 2,0).

Parte	Avaliação
Aplicação, carregamento, tipo	**Certo e coerente.** Time sharing, contíguo, multiprogramado e swapping casam com o slide 18 e com a tabela do slide 15.
Estrutura	**Não respondeu.** Esqueceu de escolher entre hierárquica, máquina virtual etc.
Política de processos (0,5)	**Parcial.** "Time-slice" está certo, mas falta nomear o algoritmo (round-robin, preemptivo) e justificar a escolha (response time em sistema interativo).
Deadlock (0,5)	**Errado.** "Liberar os recursos quando não forem mais necessários" não evita deadlock. O deadlock acontece justamente quando processos **retêm** recursos enquanto esperam outros. Nenhuma condição de deadlock é quebrada.
Starvation (0,5)	**Errado.** Escalonar **por prioridade** é justamente o que **causa** starvation: processos de baixa prioridade podem nunca rodar. Seria preciso citar o **round-robin** (rodízio garante a vez de todos) ou o **envelhecimento (aging)** das prioridades.

Parte	Avaliação
Gerência de memória real (0,5)	**Certo.** Swapping.

Resposta modelo. É uma questão aberta: qualquer projeto **coerente** é aceito. Vale mais a coerência entre as escolhas do que a escolha em si.

Escolhas de projeto:

- **Aplicação característica: time sharing.** Muitos usuários interativos em terminais. As transações são curtas e voltadas a E/S. O objetivo principal é **minimizar o response time** (slide 4 de `8-so-red-1` : "time-sharing: uso de time slice").
- **Tipo: multiprogramado.** Vários processos residentes competem pela CPU. Enquanto um espera E/S, outro executa (slide 10 de `7-teoria-1`).
- **Carregamento: contíguo** (cada programa num bloco único), coerente com a gerência escolhida abaixo. Uma alternativa igualmente válida seria carregamento não contíguo com paginação.
- **Estrutura: hierárquica (em camadas).** O nível 0 (kernel) trata interrupções e escalonamento, e cada nível usa os serviços dos inferiores. Isso dá bom nível de abstração (slide 25 de `8-so-red-2`).

Política de processos (0,5). Escalonamento **round-robin preemptivo com time slice**, opcionalmente com prioridades e envelhecimento (aging).

- **Justificativa:** em time sharing, nenhum usuário pode monopolizar a CPU. A preempção por interrupção de relógio, ao fim do time slice, garante tempo de resposta baixo e previsível para os processos interativos.
- **Coerência com a memória:** o processo preemptado volta ao fim da fila de prontos. Isso casa com o swapping, que "troca processos via interrupção (time slice)" (slide 18 de `8-so-red-2`).

Como evita deadlock (0,5). O escalonamento da CPU sozinho não evita deadlock (ver Q20). Por isso a política precisa incluir uma regra de **alocação de recursos** que quebre uma das condições necessárias do deadlock:

- **Escolha principal: ordenação dos recursos.** Os recursos são numerados, e cada processo só pode pedir um recurso de número maior que os que já possui. Isso impede a **espera circular**.
- **Alternativa válida: alocação prévia.** O processo pede todos os recursos de uma vez antes de executar, o que quebra a **posse e espera**. A desvantagem é reduzir o throughput (ver Q16).
- **Papel do S.O.:** ele controla a alocação (slide 10 de `8-so-red-1`). Se o recurso estiver ocupado, o processo é bloqueado na fila de bloqueados, e o S.O. só concede pedidos que respeitem a

regra.

Como evita starvation (0,5).

- **Rodízio:** no round-robin, todo processo pronto recebe, em rodízio, uma fatia de tempo. Com N processos na fila e fatia q, cada um espera no máximo (N−1)·q para voltar à CPU, então nenhum fica esperando indefinidamente.
- **Se houver prioridades:** usa-se **envelhecimento (aging)**. A prioridade de um processo cresce enquanto ele espera, até que ele seja escalonado.
- **Na fila de bloqueados:** os pedidos de recurso são atendidos em ordem FIFO.

Política de gerência de memória real (0,5). Swapping: multiprogramação com alocação contígua (slide 15).

- Ao fim do time slice, ou quando o processo bloqueia esperando E/S, ele pode ser levado da memória para o disco, liberando espaço para outro processo da fila de prontos (slide 18).
- O custo é alto, com tempos de remoção e carga na casa dos milissegundos. Por isso convém usar **relocação dinâmica por registrador de base** (slide 19), para que o processo possa voltar em outra posição da memória.
- Uma alternativa mais moderna seria **paginação** (não contígua), com a vantagem de não ter fragmentação externa (ver Q12).

Outras combinações coerentes que também valeriam nota:

- **Batch + multiprogramado + contíguo + partições.** O escalonamento é FIFO ou SJF não preemptivo, para maximizar o throughput. Para evitar deadlock, pede-se a alocação prévia de todos os recursos (aceitável em batch). Para evitar starvation, usa-se aging no SJF.
- **Tempo real + prioridades fixas preemptivas + memória contígua/partições fixas** (sem paginação, para ter tempos determinísticos).

Dica: são 2,0 pontos divididos em quatro partes de 0,5. Responda **cada** item pedido (aplicação, carregamento, tipo, **estrutura**, política, deadlock, starvation, memória) em tópicos separados. Lembre de duas armadilhas:

- **Prioridade sem aging causa starvation**, não evita.
- **Escalonamento da CPU não evita deadlock**: é preciso uma política de alocação de recursos.

Assuntos cobrados nesta prova

Assunto	Questões	Qtde	Conceitos-chave a dominar
Arquitetura de von Neumann (componentes, ciclo, gargalo, localidade)	O1, O2, O4, O14, D1	5	4 componentes (CPU, memória, periféricos, **barramentos**). CPU = ULA, UC, registradores, PC. Ciclo busca–decodificação–busca de operandos–execução, PC ← entry point, PC incrementado a cada busca. Gargalo CPU x memória (cache, pipelining) x gargalo memória x E/S (blocagem/bufferização, DMA, canais). Localidade temporal e espacial e suas causas.
Código absoluto x relocação / carregadores	O5, O7, O8, O15, D2	5	Carregador absoluto (ROM/não volátil, power on, carrega o S.O., sem relocação). Código absoluto ainda usado no boot (FFFF:0000). Carregador de relocação (tamanho, bloco contíguo, reloca na carga, entrypoint). Relocação viabiliza bibliotecas de código objeto (bytecode Java não é exemplo). Instantes de relocação: nenhum, carga, execução.
Montador	O6, O9, O22	3	Entrada Assembly e saída **objeto relocável**, tabela de endereços relocáveis e lista de referências externas. Pré-passo / passo 1 / passo final. Síntese: otimização **do código intermediário**, não algorítmica.
Ligador	O3, O13, O21	3	4 etapas (ler/relocar, tabela global, concatenar, resolver referências externas). Saída **executável** relocável. Tipos: Linkage Editor, Linking Loader/Binder, ligação dinâmica (DLL).
Escalonamento, deadlock, starvation, região crítica	O16, O17, O19, O20, D4	5	Objetivos (throughput ↑, response e turnaround ↓). Preemptivo x não preemptivo. Round-robin evita **starvation**, não deadlock. Alocação prévia evita deadlock mas reduz o throughput. Prioridade sem aging causa starvation. Região crítica e exclusão mútua.

Assunto	Questões	Qtde	Conceitos-chave a dominar
Gerência de memória (real e virtual), tipos de S.O.	O10, O11, O12, O18, D3, D4	6	Tabela do slide 15 (contígua x não contígua, mono x multiprogramação). Batch ↔ partições; time sharing ↔ swapping/time slice. Paginação (fixa, fragmentação interna, concatenação) x segmentação (variável, fragmentação externa, soma). Page fault → programa de canal + bloqueio. Mapeamento EL → ER com RegBase + tabela de páginas.

A questão D4 aparece em duas linhas porque cobra escalonamento e memória.

Pontos fracos identificados

1. **Mapeamento de endereços em paginação (D3).** Esse é o ponto mais fraco. O estudante confundiu relocação por registrador de base (ER = EL + RegBase) com paginação. Não dividiu o endereço em página e deslocamento, não citou a concatenação e não deu exemplo. **Treine:** fazer à mão um exemplo numérico com uma tabela de páginas.
2. **Deadlock e starvation (D4).** As justificativas estavam conceitualmente erradas. "Liberar recursos" não evita deadlock, e "prioridade" causa starvation em vez de evitar. **Estude:** as condições de deadlock (exclusão mútua, posse e espera, não preempção, espera circular), como quebrar cada uma, e o papel do round-robin e do aging.
3. **Completude das discursivas (D2, D4).** Ficaram partes pedidas sem resposta: o instante da relocação em D2 e a estrutura do S.O. em D4. **Estratégia:** transforme o enunciado num checklist e responda item por item.
4. **Itens V/F com omissão (O14).** O estudante marcou F um item que só reproduz o slide sem citar um detalhe (incremento do PC). **Regra prática:** para marcar F, aponte uma afirmação **errada** no texto, não algo que falta. A exceção são as listas definidoras, como na Q1.
5. **Síntese do montador (O6).** A marcação ficou dupla ("F/NS"), o que é inválido, e a justificativa contradiz o slide 31 ("a otimização é do código e não das funções do programador").
6. **Bytecode Java e código relocável (O15).** O estudante não sabia (NS). Entenda que relocação estática viabiliza bibliotecas de código objeto, mas que a portabilidade do bytecode vem da **JVM**.
7. **Localidade de referência (D1).** O "o que é" estava bom. Nas causas, ele repetiu os tipos (temporal e espacial) em vez das causas concretas (laços, execução sequencial, vetores contíguos, pilha).

Itens com resposta incerta (dependem da convenção do professor)

- **O14 (gabarito provável V):** o mais incerto. Se o professor considera falso um item que omite o incremento do PC, a resposta muda para F.
- **O6 (gabarito provável F):** depende de o professor cobrar a nota do slide 31 sobre otimização "não algorítmica".
- **O15 (gabarito provável F):** depende de o professor aceitar o bytecode Java como exemplo de biblioteca de código objeto.
- **O17 (gabarito provável V):** depende de ele ler região crítica como o trecho que "garante" a exclusão mútua ou como o trecho que a "exige".
- **O10 (provável F) e O13 (provável V):** os slides não trazem associação explícita entre tipo de S.O. e gerência de memória, nem o termo "carregador de função".
- **O1 (provável F):** o item é falso por omissão (faltam os barramentos), o que segue o padrão do professor.